Partie 1 : Audit Système

Le but de cette première partie est de décrire toutes les commandes nécessaires a la suite du projet.

Comment avons-nous procédé pour trouver les commandes que nous ne connaissions pas? N'étant pas super familiers avec Linux de base, il y a de nombreuses commandes que nous ne connaissions pas, avant de chercher toutes ces commandes, nous avons donc mis en place une "stratégie" qui était de chercher avec apropos un mot clé, et ensuite de chercher dans man comment l'utiliser et donc comment trouver l'information que nous voulions. Par exemple pour la version du noyau, nous avons tapé apropos kernel et nous avons vu qu'il y avait proc_version qui nous permettait de voir la version du noyau. Nous avons donc tapé man proc_version et nous avons vu que nous pouvions faire cat /proc/version. Nous avions donc notre information et cela nous a permis de mettre en place un réflexe de chercher par nous-mêmes au lieu d'aller directement chercher sur google (qui est notre solution de secours si nous ne trouvons vraiment rien).

Nom de la machine :

La commande est `hostname`. Elle retourne "debian-noa". Il n'y a pas besoin de sudo.

Distribution (nom et version):

Nous pouvons le faire avec `cat /etc/os-release` qui donne toutes les informations dont celle que l'on cherche: - PRETTY_NAME="Debian GNU/Linux 13 (trixie)" - NAME="Debian GNU/Linux" - VERSION_ID="13" - VERSION="13 (trixie)" - VERSION_CODENAME=trixie - DEBIAN_VERSION_FULL=13.7 - ID=debian

Et nous pouvons également faire ça avec `lsb_release -d` qui nous retourne: "Description: Debian GNU/Linux 13 (trixie)". On priorisera d'utiliser `/etc/os-release` car comme l'explique le sujet, la commande `lsb_release` dépend du paquet `lsb-release`, absent des installations minimales et de la plupart des images Docker. Nous n'avons pas besoin de sudo.

Version du noyau :

L'explication est détaillée au dessus dans l'exemple de notre stratégie. Cela donne :

```
noa@debian-noa:~/PreSaeNoa$ cat /proc/version
Linux version 6.12.113+deb13-amd64 (debian-kernel@lists.debian.org) (x86_64-
linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.0, GNU ld (GNU Binutils for Debian)
2.44) #1 SMP PREEMPT DYNAMIC Debian 6.12.113-1 (2026-10-15)
```

Nous avons également testé `uname -r` où `-r` donne la version du noyau en cours sur le système d'exploitation:

```
noa@debian-noa:~/PreSaeNoa$ uname -r
6.12.113+deb13-amd64
```

Nous utiliserons `uname -r` car notre sortie est pile ce dont nous avons besoin et n'est pas brute comparée à `cat /proc/version`. Nous n'avons pas eu besoin de sudo.

Uptime et charge moyenne :

Pour uptime et la charge moyenne, nous avons utilisé la commande `uptime` (qui est explicite):

```
noa@debian-noa:~/PreSaeNoa$ uptime
17:52:29 up 1:03, 3 users, load average: 0,01, 0,04, 0,01
```

Comme nous pouvons le voir, elle retourne le temps depuis lequel notre machine est démarrée, donc là ça fait 1h03 et elle retourne également la charge moyenne du système dans la dernière minute, les 5 dernières minutes et les 15 dernières minutes. On constate que notre système était plutôt tranquille.

Mémoire et swap:

Pour la mémoire et swap disponibles et utilisés, on utilise la commande `free`. Nous ne la connaissions pas avant de faire ce projet, nous l'avons découverte en testant les commandes décrites dans les commentaires de la partie 1 du sujet. En faisant `man free`, nous avons également découvert que nous pouvions accéder aux informations que l'on cherche en faisant un `cat /proc/meminfo`. Elle nous retourne ceci :

```
noa@debian-noa:~/PreSaeNoa$ free -h
               total        utilisé      libre     partagé tamp/cache
disponible
Mem:           1,9Gi       1,0Gi       489Mi       584Ki       553Mi
905Mi
Échange:       1,1Gi          0B       1,1Gi
```

Le paramètre `-h` permet d'automatiquement mettre à l'échelle la plus appropriée pour qu'un humain puisse lire facilement: - B = octets - Ki = kibioctets - Mi = mébioctets - Gi = gibioctets. Le total de mem correspond à la quantité totale de mémoire allouée à notre système (nous sommes dans une VM donc nous avions mis 2GO). Et Le total du swap correspond à la taille totale de l'espace de secours réservé sur notre disque dur de notre VM. Utilisé correspond a la mémoire utilisé ou a la mémoire indisponible. Libre correspond a la mémoire inutilisée. Partagé correspond a la mémoire utilisé surtout par tmpfs. tamp/cache correspond a la mémoire utilisé par les tampons du noyau, par les cache de page et les slabs. disponible correspond a la mémoire disponible pour le démarage de nouvelles applications sans utiliser l'espace d'échange. La différence entre libre et disponible est que la mémoire disponible est l'addition de la mémoire libre et de la mémoire temporairement utilisée car elle peut être récupéré s'il le faut. Concernant le swap (échange), nous avons fait des recherches sur google cette fois-ci, et nous avons vu qu'il n'avait tout simplement pas de système de cache et donc pas de colonne disponible. Nous n'avons pas eu besoin de sudo.

Liste des interfaces réseau (adresses MAC et IP v4/v6):

```
noa@debian-noa:~/PreSaeNoa$ ip addr show
1: lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
```

```
default glen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST, MULTICAST, UP, LOWER UP> mtu 1500 qdisc fq_codel state
UP group default qlen 1000
    link/ether 08:00:27:12:33:59 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute
enp0s3
       valid_lft 74276sec preferred_lft 63476sec
    inet6 fd17:625c:f037:2:a00:27ff:fe12:3359/64 scope global dynamic
mngtmpaddr proto kernel_ra
       valid_lft 86268sec preferred_lft 14268sec
```

On constate que l'adresse mac de enp0s3 est celle de notre machine virtuelle car on a le préfixe de VirtualBox: 08:00:27. Nous n'avons pas besoin de sudo.

Liste des utilisateurs humains existants :

Nous pouvons deja voir tous les comptes de la machine grâce a la commande `cat /etc/passwd`. Ensuite, grâce a la commande `cat /etc/login.defs`, nous pouvons obtenir l'UID MIN et l'UID MAX, qui sont respectivement 1000 et 60000, c'est à dire que l'uid d'un utilisateur humain ne peut que être compris entre 1000 et 60000, ainsi lorsque nous cherchons dans la sortie de `cat /etc/passwd`, nous pouvons apercevoir qu'il y a que l'utilisateur noa qui est un utilisateur humain :

```
noa:x:1000:1000:noa,,,:/home/noa:/bin/bash
```

Nous pouvons également vérifier cela grâce a la commande `who` ou `users`:

```
noa@debian-noa:~$ users
noa noa
```

Nous n'avons pas eu besoin de sudo.

Espace disque par partition :

Pour obtenir l'espace disque par partition, nous avons lancé `df -h` afin d'avoir des données compréhensibles.

```
noa@debian-noa:~/PreSaeNoa$ df -t ext4 -h
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
/dev/sda1          7,4G    1,8G  5,3G  25% /
/dev/sda6           12G    1,4G  9,2G  14% /home
```

On a en sortie uniquement les vraies partition de disque. Nous n'avons pas eu besoin de sudo.

Les N processus les plus consommateurs de CPU :

Nous avons utilisé `ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -n N`.

```
noa@debian-noa:~/PreSaeNoa$ ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -n
```

```
5
PID %CPU %MEM COMMAND
1324 1.2 22.3 MainThread
1114 1.0 1.1 code-6d62b1d5da
1129 0.7 6.1 MainThread
1295 0.3 1.1 code-6d62b1d5da
```

On peut ajouter `--no-headers` pour enlever les noms de colonnes. Nous n'avons pas eu besoin de sudo.

Liste des connexions réseau avec le processus associé :

En cherchant dans `man ss`, nous avons trouvé notre commande qui est `ss -tuapn`:

```
noa@debian-noa:~/PreSaeNoa$ sudo ss -tuapn
Netid State Recv-Q Send-Q Local Address: Port Peer Address: Port Process
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=709,fd=6))
tcp ESTAB 0 0 10.0.2.15:22 192.168.1.13:51770 users:(("sshd-
session",pid=1276,fd=7),("sshd-session",pid=1266,fd=7))
```

A savoir: le sujet demande que le script Ruby devra tourner sans root, donc là où le processus est vide, il faudra l'indiquer comme "inaccessible" sans planter. C'est pour cela que dans l'exemple ci-dessus, on ajoute sudo pour tout voir.

Présence et statut de certains services clés (ex.: ssh, cron, docker) :

Pour ces 3 services, nous avons utilisé `systemctl is-active` et `systemctl is-enabled`. Comme le sujet en parle, nous avons également testé ssh.socket:

```
noa@debian-noa:~/PreSaeNoa$ systemctl is-active ssh.socket
inactive
noa@debian-noa:~/PreSaeNoa$ systemctl is-enabled ssh.socket
disabled
```

Nous n'avons pas eu besoin de sudo.

Partie 2: Automatisation avec un script Ruby

Le script s'appelle `audit.rb`. Il regroupe toutes les informations de la partie 1, sans dialogue interactif, et n'utilise que la bibliothèque standard de Ruby: `optparse` pour les options, `json` pour l'export et `open3` pour lancer les commandes. Il n'y a aucune gem à installer, ce qui simplifie aussi l'image Docker de la partie 3.

Dépôt git: https://github.com/camil-1/projet_tuteure_craincourt_taleb (dépôt public).

Utilisation

```
ruby audit.rb -f texte # affichage lisible
ruby audit.rb # JSON sur la sortie standard
ruby audit.rb -n 10 -s ssh,cron,docker,nginx # 10 processus, services choisis
ruby audit.rb -o audit.json # JSON enregistré dans un fichier
```

```
ruby audit.rb -r /host -f texte # fichiers système lus sous /host
```

Option	Rôle	Défaut
`-n, --nombre N`	nombre de processus affichés (CPU et mémoire)	5
`-f, --format`	json ou texte	json
`-s, --services`	liste de services séparés par des virgules	ssh,cron,docker
`-r, --racine`	répertoire sous lequel lire `/etc`, `/proc`..	/
`-o, --sortie`	enregistre le JSON dans ce fichier (prioritaire sur `-f`: rien n'est affiché à l'écran)	aucun

Limites et contrôles des paramètres dans le script

L'option `-n` (nombre de processus) est typée via optparse (`Integer`), ce qui fait échouer le script si l'utilisateur entre une chaîne de caractères. En revanche, le script actuel ne vérifie pas les autres entrées. Une valeur négative pour `-n` n'est pas contrôlée et provoque une exception non rattrapée (`ArgumentError` dans take). Un format inconnu pour `-f` basculera silencieusement sur le format JSON. Les noms de services fournis via `-s` sont insérés tels quels dans la commande `systemctl` lancée par `Open3.capture3` sous forme de chaîne unique. Si le nom contient des caractères spéciaux du shell (par exemple `;`), la commande est interprétée par un shell: c'est une injection de commande possible, et pas seulement une erreur. Une correction simple consiste à passer les arguments séparément (`capture3("systemctl", "is-active", unite)`) ou à valider les noms avec une expression régulière. Enfin, l'option `-o` est prioritaire sur `-f`: avec les deux, le JSON est écrit dans le fichier et rien n'est affiché.

Organisation du code

Chaque information a sa méthode (`audit_machine`, `audit_uptime`, `audit_memoire`, `audit_reseau`, `audit_utilisateurs`, `audit_disque`, `audit_processus`, `audit_connexions`, `audit_services`). Elle renvoie un Hash, ou un tableau de Hash pour les listes. Le résultat global est un Hash qui contient la date de l'audit (`Time.now.strftime('%Y-%m-%d %H:%M:%S')`), les paramètres utilisés et le retour de chaque méthode. Le JSON a donc ces clés de premier niveau : `date_audit`, `parametres`, `machine`, `uptime_et_charge`, `memoire`, `reseau`, `utilisateurs`, `disques`, `processus_gourmands`, `connexions`, `services_cles`.

Ce qui change par rapport à la partie 1

Dans le script, certains choix sont différents, pour deux raisons: l'option `--racine` oblige à lire des fichiers plutôt qu'à interroger des commandes (lorsque cela est pris en compte par la commande), et un script a besoin d'une sortie facile à analyser.

Information	Partie 1	Script	Pourquoi
Nom de la machine	`hostname`	`/etc/hostname` sous	`hostname` interroge le

Information	Partie 1	Script	Pourquoi
		la racine	conteneur, pas l'hôte. Si le fichier manque, le script n'appelle pas la commande: le champ contient "Erreur: Fichier introuvable".
Distribution	`cat /etc/os-release`	même fichier, lu avec `PRETTY_NAME`	pas de dépendance à `lsb-release`
Noyau	`uname -r`	`uname -r`	sortie propre, et c'est la même valeur dans un conteneur (noyau partagé)
Uptime et charge	`uptime`	`/proc/uptime` et `/proc/loadavg`	`uptime` lit ces fichiers et les met en forme. Les lire directement évite d'analyser du texte formaté (le séparateur décimal change avec la langue).
Mémoire et swap	`free -h`	`/proc/meminfo`	même source que `free`. Utilisée = MemTotal - MemAvailable, comme `free`.
Réseau	`ip addr show`	`ip addr show`	le script utilise des expressions régulières pour isoler les adresses MAC, IPv4 et IPv6 de la sortie brute. L'IPv6 est présente dans le JSON, mais le mode texte n'affiche que l'IPv4.
Utilisateurs	`/etc/passwd`, `who`, `users`	`/etc/passwd` filtré par `UID_MIN` et `UID_MAX`, puis `users`	utilisation de la commande directe `users` dans le script
Disques	`df -h`	`df -P`	le script lance `df -P` et se contente de filtrer les lignes qui commencent par `^/dev/`
Processus	`ps -eo pid,%cpu,%mem,comm --sort=-%cpu`	même commande avec `--no-headers`	le script garde les N premières lignes

Information	Partie 1	Script	Pourquoi
Connexions	`ss -tuapn`	`ss -tuapn`	le script ignore manuellement la première ligne (en-tête)
Services	`systemctl is-active` et `is-enabled`	idem, plus `ssh.socket` quand on teste `ssh`	sur Ubuntu récent, SSH peut être démarré par la socket

Limites du paramètre `--racine` dans le script: Dans la version actuelle du script, l'option `--racine` ne modifie le chemin de lecture que pour l'audit de la machine (`/etc/hostname`, `/etc/os-release`), de la mémoire (`/proc/meminfo`), de l'uptime/charge (`/proc/uptime`, `/proc/loadavg`) et des utilisateurs humains (`/etc/passwd`, `/etc/login.defs`). Cependant, les commandes d'audit réseau (`ip`), des disques (`df`), des processus (`ps`), des connexions (`ss`), de la session de l'utilisateur connecté (`users`) et des services (`systemctl`) ignorent ce paramètre et s'exécutent avec leur contexte natif. Cette limite a un impact direct sur les résultats dans la partie Docker.

Disques et filtrage: Le script lance `df -P` et filtre les lignes avec la regex `^/dev/`. Il est important de noter que ce filtre fonctionne bien sur une Debian pure, mais sur Ubuntu, il listera également toutes les partitions virtuelles `/dev/loop*` créées par les paquets Snap, ce qui pollue la sortie. C'est une limite technique assumée du script.

Connexions. Sans root, `ss -p` ne montre que les processus de l'utilisateur courant. Le script distingue trois cas. Si le processus est connu, il affiche son nom et son PID. Si l'on n'est pas root (`Process.uid != 0`) et que le processus manque, il écrit « inaccessible sans root ». Si l'on est root et que le processus manque, c'est qu'aucun processus n'est associé au socket, par exemple une connexion en TIME-WAIT.

Robustesse

Le sujet impose que le script tourne sans root et ne s'arrête jamais sur une erreur. Voici comment il y répond: - `executer_commande` lance les commandes avec `Open3.capture3`. Elle renvoie la sortie standard, la sortie d'erreur et le code de retour. Une commande absente ou en échec produit un message d'erreur dans le résultat, pas une exception bloquante (en revanche, elle ne protège pas contre l'injection de commande décrite plus haut). - L'utilisation de `File.exist?` permet de vérifier localement la présence des fichiers, et chaque méthode gère le cas où un fichier manque. - Les informations qu'on ne peut pas lire sans root (comme les PID via `ss`) sont signalées comme telles ("inaccessible sans root"). Cette robustesse a des limites: une valeur négative pour `-n` fait planter le script, et si `ps` échoue, le mode texte affiche une ligne vide (`PID | CPU: %`) au lieu du message d'erreur, alors que le JSON contient bien l'erreur.

Partie 3: Déploiement dans Docker

Le Dockerfile

L'image part de `debian:stable-slim` et installe uniquement ce que le script utilise: `ruby`, `iproute2`, `procps`, `systemd`. L'installation utilise `--no-install-recommends`, et `/var/lib/apt/lists` est supprimé dans la même couche pour garder l'image légère. Le point d'entrée est `ENTRYPOINT ["ruby", "/app/audit.rb"]` sous la forme exec: tout ce qui suit le nom de l'image dans `docker run` est transmis au script.

Options retenues pour auditer l'hôte

Le fichier `compose.yaml` regroupe toutes les options nécessaires pour auditer l'hôte malgré l'isolation du conteneur.

Option	**Information concernée**	**Justification**
`pid: host`	processus, et `/proc/<pid>` pour `ss -p`	le conteneur partage l'espace de noms PID de l'hôte
`network_mode: host`	interfaces et connexions	`ip` et `ss` voient la vraie pile réseau
`/:/host:ro` avec `--racine /host`	`/etc`, `/proc`, montages	l'hôte entier est visible en lecture seule.
`/run/systemd:/run/systemd:ro`	services	contient la socket de systemd
`SYSTEMD_IGNORE_CHROOT=1`	services	avec `pid: host`, `systemctl` se croit dans un chroot, il faut forcer son comportement
`cap_add: SYS_PTRACE`	processus des connexions	lire `/proc/<pid>/fd` d'un autre utilisateur demande cette capacité
`apparmor=unconfined`	processus des connexions	le profil `docker-default` interdit le ptrace
`no-new-privileges:true`	sécurité	empêche toute élévation de privilèges
`read_only: true` et `tmpfs: /tmp`	sécurité	verrouille le système de fichiers de l'image
`./out:/out`	export JSON	seul dossier accessible en écriture, pour enregistrer le fichier

Comparaison entre l'hôte et le conteneur

Nous avons lancé le script sur l'hôte sans root, sur l'hôte avec sudo (pour valider ce que le conteneur voit) et dans le conteneur avec docker compose, mais aussi le conteneur seul sans option (pour voir le comportement natif de l'isolation Docker).

Information	Hôte (sans root)	Hôte (avec sudo)	Conteneur (avec options)	Conteneur (sans options, natif)	Explication (et impact des limites du script)
Nom de la machine	camil	camil	identique	28b11c92eb71	Avec option, `--racine` modifie bien le chemin vers `/host/etc/hostname`. Sans option, Docker génère son propre `hostname` basé sur l'ID du conteneur.
Distribution	Debian 13 (trixie)	Debian 13	identique	Debian 13 (trixie)	L'image de base (`debian:stable-slim`) est la même que l'hôte, d'où le résultat identique. Si l'hôte était sur Ubuntu, la différence serait visible sans option.
Noyau	6.12	6.12	identique	6.12	Le noyau est toujours partagé entre l'hôte et les conteneurs (via `uname -r`), d'où le résultat identique même sans option.
Date	2026-10-23 16:44:26	2026-10-23 16:45:00	2026-10-23 14:45:10	2026-11-03 18:44:04	Hôte en UTC+2, Conteneur (avec ou sans option) en

Information	Hôte (sans root)	Hôte (avec sudo)	Conteneur (avec options)	Conteneur (sans options, natif)	Explication (et impact des limites du script)
					UTC (le format de strftime cache le fuseau mais l'heure diffère).
Mémoire	RAM lue sur l'hôte	RAM lue sur l'hôte	identique	identique	`--racine` permet de lire `/host/proc/meminfo`. Sans option, `/proc/meminfo` n'est pas virtualisé par défaut par Docker, il affiche donc la RAM totale de la machine hôte.
Utilisateurs	Connecté	Connecté	Connecté	Aucun compte (Objet vide `{}`)	**Avec options :** L'option `-r /host` permet de bien trouver le compte dans `/host/etc/passwd`, et la commande `users` exécutée nativement (aidée par le fait que l'on partage l'espace pid host ou que l'on ait monté certaines parties) réussit à détecter la

Information	Hôte (sans root)	Hôte (avec sudo)	Conteneur (avec options)	Conteneur (sans options, natif)	Explication (et impact des limites du script)
					session de l'hôte. **Sans options :** le compte n'est même pas trouvé car le script lit le `/etc/passwd` du conteneur.
Disques	`/` et `/boot/efi`	`/` et `/boot/efi`	Préfixes `/host` ajoutés	Seulement le point de montage que Docker lui a alloué (`/etc/hosts`)	Limite du script: `df -P` ignore le paramètre racine. Le conteneur avec options liste tous les disques montés sous le préfixe `/host` ajouté par notre `compose.yml`. Sans option, le conteneur n'a pas de disques à lui, seulement l'`overlay` et les fichiers que Docker y monte.
Connexions	10 inaccessibles sans root	Tous visibles (avahi, cupsd..)	Tous visibles	Aucune connexion listée `[]`	Le conteneur sans option est isolé au niveau réseau. Les options (`network_mode: host` et `SYS_PTRACE`) permettent au

Information	Hôte (sans root)	Hôte (avec sudo)	Conteneur (avec options)	Conteneur (sans options, natif)	Explication (et impact des limites du script)
					conteneur, tournant en root, de remonter les mêmes connexions et PID que l'hôte avec sudo.
Processus gourmands	ruby 85%, etc.	ruby 85%, etc.	docker-compose, ruby..	Uniquement `ruby` (PID 1) et `ps` (PID 24)	`ps` ignore la racine, mais avec `pid: host`, il lit les bons processus de l'hôte. Sans option, on voit bien l'isolation de namespace PID: le conteneur ne voit que son propre processus et la commande exécutée.
Services	cron et docker actifs et activés, ssh non installé	identique	identique	Tous les services renvoient l'erreur : "System has not been booted with systemd as init system"	Sans monter la socket systemd (et sans `SYSTEMD_IGNORE_CHROOT`), la commande `systemctl` échoue lamentablement dans le conteneur car le PID 1 n'est pas systemd, mais ruby.

Implications en matière de sécurité

Ces options rendent le conteneur beaucoup moins isolé que d'habitude. Voici ce qu'un attaquant qui prendrait le contrôle de ce conteneur pourrait faire: - `/:/host:ro` expose tous les fichiers de l'hôte en lecture. Le root du conteneur peut lire `/etc/shadow`, les clés SSH privées. - `network_mode: host` supprime l'isolation réseau. Le conteneur peut écouter sur les ports de l'hôte, ou capturer du trafic réseau local. - `apparmor=unconfined` retire les garde-fous par défaut du moteur Docker. - `/run/systemd:ro` n'empêche pas d'envoyer des ordres à systemd. Une socket Unix se connecte même sur un montage en lecture seule. Comme le root du conteneur a l'UID 0, systemd lui obéit. Il pourrait arrêter ou créer des services sur l'hôte, ce qui revient à en prendre le contrôle.

Les options `no-new-privileges`, `read_only` limitent les dégâts, mais ne compensent pas les accès précédents. Pour un audit ponctuel sur sa propre machine, le risque est acceptable. Pour une automatisation, lancer le script directement sur l'hôte via cron est préférable.